

NittanyAuction Phase I Report

Team Members: Anuraj Pahwa, Sean Hogan, Tyler
Miller, and Dylan Burkholder

Table of Contents

NittanyAuction Phase I Report	i
Table of Contents	i
List of Figures	iii
List of Tables.....	Error! Bookmark not defined.
1. Introduction	1
2. Requirement Analysis	2
System Overview	2
User Roles.....	3
Product Listing Requirements.....	12
Bidding Rules and Constraints.....	14
Category Hierarchy	15
Proposed New Feature: Product Watchlist and Auction Notifications	15
3. Conceptual Database Modeling.....	17
Users: Bidders, Sellers, and HelpDesk.....	18
Products, Bidding, Categories	19
Notifications and Watchlist	20
Additional Notes	20

4. Technology Survey	22
Overview of Current Web and Database Technologies	22
The Baseline Stack: Flask, Python, PyCharm, and SQLite.....	23
Alternative Stack: Django, Python, PostgreSQL	24
Alternative Stack: MongoDB, Express, React, Node.js.....	25
Alternative Stack: React/Next.js, Node.js, PostgreSQL	26
Database Managemenet Systems Comparison.....	27
Development Tools and IDEs	28
Final Recommendation	28
 5. Logical Database Design and Schema Normalization	 32
Overview	32
Mapping the ER Diagram to Relations	33
Initial Relational Schema Before Normalization.....	38
Functional Dependencies	39
Normalization.....	39
Final Normalized Schema	41
SQL Schema.....	42
Summary	45

List of Figures

Figure 1 – A complete ER diagram.....	17
---------------------------------------	----

1. Introduction

This document contains a full description of the way our team has interpreted NittanyAuction. This includes a description (as understood by our team) of each aspect of the system. This also includes our proposed feature, the watchlist, which allows users to stay notified of updates regarding certain products. Following this is the resulting ER diagram and associated description. Finally, we create a logical schema from all the previous information.

NittanyAuction is a web-based marketplace designed to support the buying and selling of goods through an auction system within a university community. The platform focuses primarily on students and approved local vendors who wish to exchange products in a convenient online environment. By providing an auction-based model, the system allows sellers to receive competitive offers for their items while allowing buyers to bid on products they are interested in purchasing.

The platform operates through a centralized database that manages all information related to users, product listings, bids, and transactions. By managing this information in a single database, the system can keep track of auction activity and maintain accurate records as users interact with the platform. This structure also allows multiple users to access and use the system at the same time while ensuring that the data remains consistent and reliable.

2. Requirement Analysis

2.1 System Overview

Users interact with NittanyAuction through a web interface that provides access to various system functions. These functions include creating accounts, browsing available products, placing bids, and managing personal information. The system differentiates between types of users based on their roles. Standard users participate as bidders, while some users may also become sellers who can list products for auction. In addition, administrative staff members are responsible for overseeing certain operations of the system and managing requests that require manual approval.

The auction process is the central component of the platform. Sellers create listings for products by specifying details such as the item description, category, reserve price, and auction duration. Once a listing becomes active, other users can view the product and submit bids. The system records each bid and ensures that new bids exceed the current highest bid by a defined increment. When the auction period ends, the bidder with the highest valid offer becomes the winner of the auction.

To help users locate items efficiently, the system organizes products using a hierarchical category structure. Categories group similar items together and allow users to narrow their searches by browsing through different levels of classification. This improves usability by making it easier for bidders to find items that match their interests.

The platform also keeps records of past auctions and transactions. Keeping this information allows the platform to track previous activity and maintain a history of how users interact with the marketplace. This history can also be used to support features such as seller ratings and to better understand how the platform is being used over time, which may help guide future improvements to the system.

Overall, NittanyAuction is intended to provide a structured online marketplace where members of the university community can buy and sell items through an auction system. By including features such as user accounts, product listings, bidding, and administrative support, the platform creates an organized environment that helps users participate in auctions more easily within the university community.

2.2 User Roles

All users must first create an account in the system. Each registered user is uniquely identified by an email address, which is the user's login identifier. During registration, users must also create a password that will be used for authentication when signing into the system. Once an account is created, the user can log in to the platform and access the features available to their assigned role.

The system maintains a set of information associated with each registered user. This information allows the platform to identify users, manage their accounts, and support transactions

within the marketplace. While different types of users may require additional data depending on their role, the basic account information is required for all registered users.

After logging into the system, users can update parts of their personal information and manage their account settings. However, some information such as the user's identifier may be restricted to maintain system consistency. Requests to change certain account details may require administrative user approval.

Once users are registered, they interact with the platform according to their assigned role. The following sections describe the three main roles within the NittanyAuction system: bidders, sellers, and HelpDesk staff.

2.2.1 Bidders Requirements

Bidders represent the primary group of users interacting with the NittanyAuction platform. These users interact with the system by browsing product listings and placing bids on items they wish to purchase. In this design of the platform, all students of Lion State University are considered as bidders once they create an account within the system.

Each bidder account stores personal information that helps identify the user and maintain their account within the system. This information includes the bidder's name, contact details, and address. Additional demographic information such as the user's academic major, age, and annual income may also be stored. This information is collected to support market analysis of activity on the platform. To maintain consistent records, each bidder is uniquely identified by an email address

which is the user's login identifier. The email attribute is a key constraint, ensuring that no two bidder accounts share the same identifier in the system.

After signing into the platform, bidders can manage certain parts of their account information. Users may update fields such as their contact details, address, and other personal information through the system interface. However, the email address cannot be modified by the user in order to preserve the integrity within the database. Any modification to the user identifier requires administrative approval. Bidders are also able to change their account password through the account management interface. Bidders may store and manage their payment information, such as credit card details, which are used when completing purchases after winning an auction. These payment records are associated with the bidder's account through a foreign key relationship linking payment information to the user identifier.

To participate in auctions, bidders must first select an item that interests them. The system provides ways for locating products through both browsing and searching. Products are organized within a hierarchical category structure that groups related items together, allowing users to search through categories and subcategories to narrow down their search. Bidders may also search for listings using keywords related to the product title or description. These features allow bidders to efficiently locate items on the platform.

Once a bidder finds a product they wish to purchase, they must submit a bid for the listing. Each bid placed by a bidder is recorded by the system and must satisfy integrity constraints before

it is accepted. Each bid is associated with a unique identifier that functions as a primary key for the bid record, while the bidder identifier and product listing identifier act as foreign keys linking the bid to the user and auction listing. A bid must be submitted before the auction's closing time and must exceed the current highest bid by at least the required increment defined by the platform. The system evaluates incoming bids to ensure these conditions are met before inserting the bid record into the database.

Bidders may place multiple bids on the same auction as long as each new bid satisfies the bidding constraints. However, a bidder may not place bids on items that they themselves have listed for sale. This constraint ensures fairness in the auction process. The system records the timestamp of each bid to maintain a clear bidding history for every auction.

When an auction reaches its end time, the bidding process closes and the bidder who submitted the highest valid bid becomes the winner, if the bid meets or exceeds the seller's reserve price. If the reserve price is not met, the seller can choose to not complete the sale and the auction would be marked as unsuccessful. For successful auctions, the winning bidder must complete the payment process through the platform after the auction concludes. The transaction record is linked to both the bidder and the auction listing through key relationships to ensure that payment activity can be tracked within the system.

After the transaction is finalized, the bidder can provide feedback about the seller. This feedback is given using a rating system that allows bidders to evaluate their experience with the

seller. Each rating submitted by a bidder is stored in a Rating entity that represents feedback associated with a completed transaction. The rating is linked to the UserID of the bidder, connecting the score to the specific user who submitted the feedback.

The rating record is linked to both the bidder and the seller involved in the transaction. The UserID of the seller and the UserID of the bidder act as foreign key constraints that reference the user records in the User entity. These relationships make sure that ratings can be traced back to the original auction and participants. To maintain consistency in the evaluation system, the rating value will follow a domain constraint restricting the rating score to an integer value between 1 and 5, representing the five-star rating scale used on the platform. An additional integrity constraint ensures that each completed auction can receive at most one rating from the winning bidder. This prevents multiple feedback entries from being submitted for the same transaction.

Ratings associated with a seller can be aggregated by the system to calculate an overall reputation score. This information may be displayed on the seller's profiles and product listings so that bidders can evaluate the reliability of a seller. Maintaining these records allows the platform to promote transparency within the marketplace.

Some bidders may wish to sell their own products on the platform. In order to do this, the bidder must first apply to become a seller. The system provides an application form where the bidder can submit the request. This request is forwarded to the HelpDesk staff for review. The application request is stored in the database and is associated with the applicant through their

UserID. Once the HelpDesk approves the request, the user's role within the system is updated to include seller privileges. This process helps maintain administrative control over who is allowed to list products for auction.

2.2.2 Sellers Requirements

Sellers are users who are allowed to publish product listings and start auctions within the NittanyAuction platform. A seller must be a registered user of the system and must log into the platform using their UserID along with their account password. After authentication, sellers are able to access additional system functionality to manage product listings and participate in the marketplace as vendors.

The system maintains additional information for each seller account beyond the basic user data stored in the User entity. This information is necessary to support financial transactions and payments associated with successful auctions. Each seller record stores banking information including a bank routing number, bank account number, and the seller's current balance. These attributes allow the platform to transfer funds from completed transactions to the seller once a buyer has successfully paid for a winning bid.

In addition to student sellers, the platform may also support local business vendors. These vendors are a specialized type of seller and require additional information. Vendor-specific attributes include the business name, business address, and phone number. These attributes allow buyers to identify the business managing the product listing and provide a method of contact if

needed. If a local business vendor decides to stop participating in the platform, the system must remove the vendor's products from the marketplace. All product listings associated with the vendor are automatically removed. This makes sure that products offered by vendors who are no longer active do not continue to appear in the marketplace. The relationship between vendor sellers and their product listings is enforced through a foreign key constraint referencing the seller's UserID, allowing the system to identify and remove the product records when a vendor account is deactivated.

After logging into the system, sellers are able to manage their account information and update certain fields related to their profile. However, similar to other user roles, the UserID cannot be modified directly by the seller. Any request to modify this identifier must be handled through the HelpDesk staff.

One of the primary responsibilities of a seller is creating product listing for auctions. To publish a listing, the seller must provide information about the product including product title, description, reserve price, and the auction closing time. The reserve price is the minimum acceptable price the seller is willing to sell the item for. Although the reserve price is stored in the system, it is not visible to bidders during the auction.

Each product listing is associated with exactly one seller and this relationship is maintained through a foreign key constraint referencing the seller's UserID. This makes sure that every

product in the system can be traced back to the seller who created it. A seller may create multiple listings and auction several products simultaneously.

Sellers are able to monitor the progress of auctions for the products they have listed. Through the interface, sellers can view all information related to their active listings, including the product details, the current highest bid, and the full bidding history of the item. In addition, bidders may submit questions regarding a product listing. These questions are stored within the system and are associated with the specific product listing and seller through key relationships. Sellers can view and respond to these questions, which helps facilitate communication between buyers and sellers.

Sellers can also view the ratings that have been submitted by bidders after completed transactions. These ratings are associated with the seller through the seller's UserID, which acts as a foreign key constraint linking the rating record to the seller account.

When creating a product listing, the seller must assign the item to an existing product category within the system. Categories are used to organize products and allow bidders to browse listings more efficiently. If a seller wishes to list a product that does not fit into any of the existing categories, the seller must submit a request to the HelpDesk staff for a new category to be created. The HelpDesk reviews the request and determines whether the new category should be added.

When an auction reaches its closing time, the system determines whether the auction was successful based on the bidding results and the seller's reserve price. If the highest bid meets or

exceeds the reserve price, the auction is successful. After the winning bidder completes the payment process, the system transfers the payment amount to the seller's account balance. If the reserve price is not met, the seller is not required to complete the sale.

Once an auction ends, the product listing is removed from the set of active listings. However, the platform keeps the auction record and bid history in the database. Maintaining these records allows NittanyAuction to analyze activity and support administrative tasks performed by the HelpDesk staff.

2.2.3 HelpDesk Requirements

HelpDesk staff members are responsible for managing administrative operations within the platform. These users make sure that the system operates correctly and that requests submitted by users are reviewed and processed. They must log into the system using their UserID along with their password. Once logged in, HelpDesk staff have access to administrative tools that allow them to review requests and manage system data.

One of the primary responsibilities of the HelpDesk team is to manage requests related to user accounts. For example, registered users are not allowed to directly modify their UserID, since it is a key identifier in the database. If a user needs to change their UserID, they must submit a request to the HelpDesk staff. The request is reviewed by an administrator, and if approved, the update is performed while maintaining referential integrity between related records in the system.

HelpDesk staff are also responsible for processing applications submitted by bidders who wish to become sellers. When a bidder submits an application form, the request is stored in the system and associated with the applicant through the user's UserID. HelpDesk staff review these applications and determine whether the user should be granted seller privileges. Once approved, the system updates the user's role to a seller.

Another responsibility for the HelpDesk team is to manage the category hierarchy used to organize products. Sellers are only allowed to list products within existing categories. If a seller wishes to list a product that does not fit within the current categories, the seller must submit a request to create a new category. The HelpDesk staff review these requests and determine if the category should be added into the system. If the request is approved, the HelpDesk creates the new category and places it within the appropriate level of the category hierarchy.

HelpDesk staff may also perform analytical tasks using system data. For example, the platform may generate reports that show the highest number of bids or the greatest auction activity over a given time period. By examining this information, the HelpDesk team can identify trends within the marketplace. These insights help the platform promote certain categories or highlight trending products on the homepage.

2.3 Product Listing Requirements

Product listings represent the items that sellers place on the platform for auction. Each product listing is stored as a record in the Product entity. Every product is uniquely identified by a ProductID, which acts as a primary key constraint for the entity. This makes sure that each product listing can be referenced and managed within the system. Along with this identifier, the system stores several attributes describing the product, including the product title, description, category, reserve price, and auction stop time.

Every product listing must be associated with exactly one seller. This relationship is enforced through a foreign key constraint referencing the seller's UserID. Although a seller may create multiple product listings, each listing corresponds to only one seller account.

Products must also be assigned to a category within the system. Each product belongs to exactly one category, and this relationship is maintained through a foreign key referencing the corresponding category record. This ensures that products appear in the correct section of the category hierarchy when bidders browse the marketplace.

Once an auction concludes, the product listing is removed from the set of active listings visible to bidders and information about the auction remains stored within the system.

2.4 Bidding Rules and Constraints

Bidding allows users to compete for products listed on the platform. Each bid submitted by a bidder is stored in the system and represents an offer to purchase a specific product listing. Every bid is associated with both the bidder and the product listing through foreign key constraints referencing the User and Product entities. These relationships make sure that every bid corresponds to a valid user and an existing auction listing.

To maintain fairness, several integrity constraints are enforced by the system. A bidder may place multiple bids on a product listing, but a seller is not allowed to place bids on their own product. In addition, each new bid must be at least \$1 higher than the current highest bid for the listing. If a submitted bid does not satisfy this, the system rejects the bid.

Bids also must be submitted before the auction's stop time. Once the auction reaches its closing time, the system stops accepting new bids and determines the highest valid bid. If the highest bid meets or exceeds the seller's reserve price, the bidder who submitted that bid wins the auction. Otherwise, the auction is unsuccessful.

2.5 Category Hierarchy

The platform organizes products using a hierarchical category structure to make it easier for bidders to browse listings. Categories represent groups of related products and allow users to narrow down their search. Every product listed on the platform must belong to exactly one category. In addition to browsing through categories, the system also provides a search feature that allows users to enter keywords and locate products directly by matching titles or descriptions.

Each category is stored in the Category entity and is uniquely identified by its category name, which acts as the key constraint. A higher-level category may contain several more specific subcategories. For example, a general category such as Electronics may contain subcategories like Laptops and Phones. In this structure, Electronics would be considered as the parent category. Categories at the lowest level of the hierarchy do not have any further subcategories.

2.6 Proposed New Feature: Product Watchlist and Auction Notifications

Bidders should easily be able to add products to a watchlist, being informed of every new bid as its made, rather than simply getting notified once an auction has concluded. This feature is useful for anybody who shows great interest in a certain product and wants to secure it for themselves. By being aware of activity on a desired product, counterbids can be entered sooner, and we can expect a higher final bid than without this feature.

Product Watchlist notifications should be included alongside the expected end-of-bid notifications in a table on a bidder's profile. These will be associated with key constraint NID, or

notification ID, to differentiate them. Notifications will also include a message string (i.e. “Another bid has been placed on [bid ID]!”) to let the user know what the notification is about, and a timestamp to let the user know when the notification was sent out. Notifications can be cleared by hitting the “X” button next to the notification, after which they will be deleted and not shown again. Only for notifications sent to the winner of a bid upon an auction’s conclusion, a hyperlink to the auction page will be attached. By following the link, the user can complete the transaction and leave a rating for the seller.

In order to subscribe to a watchlist in the first place, users must click on the “Add to Watchlist” button that appears on the page for each product. Users can remove themselves from a watchlist by hitting the “Remove from Watchlist” button. If the user attempts to join a watchlist they are already a part of, or leave one they aren’t, the request will be rejected. If a product is no longer publicly available (either from being deleted by a vendor or being archived by a seller) all users currently watching it will be removed from the watchlist. If a user is removed, all of their watches relationships are removed.

3. Conceptual Database Modeling

See Figure 1 below for the entire ER diagram. Since the diagram is very large, zooming in may be required to effectively read the names. Following that is the narrative description, broken up by notable entities.

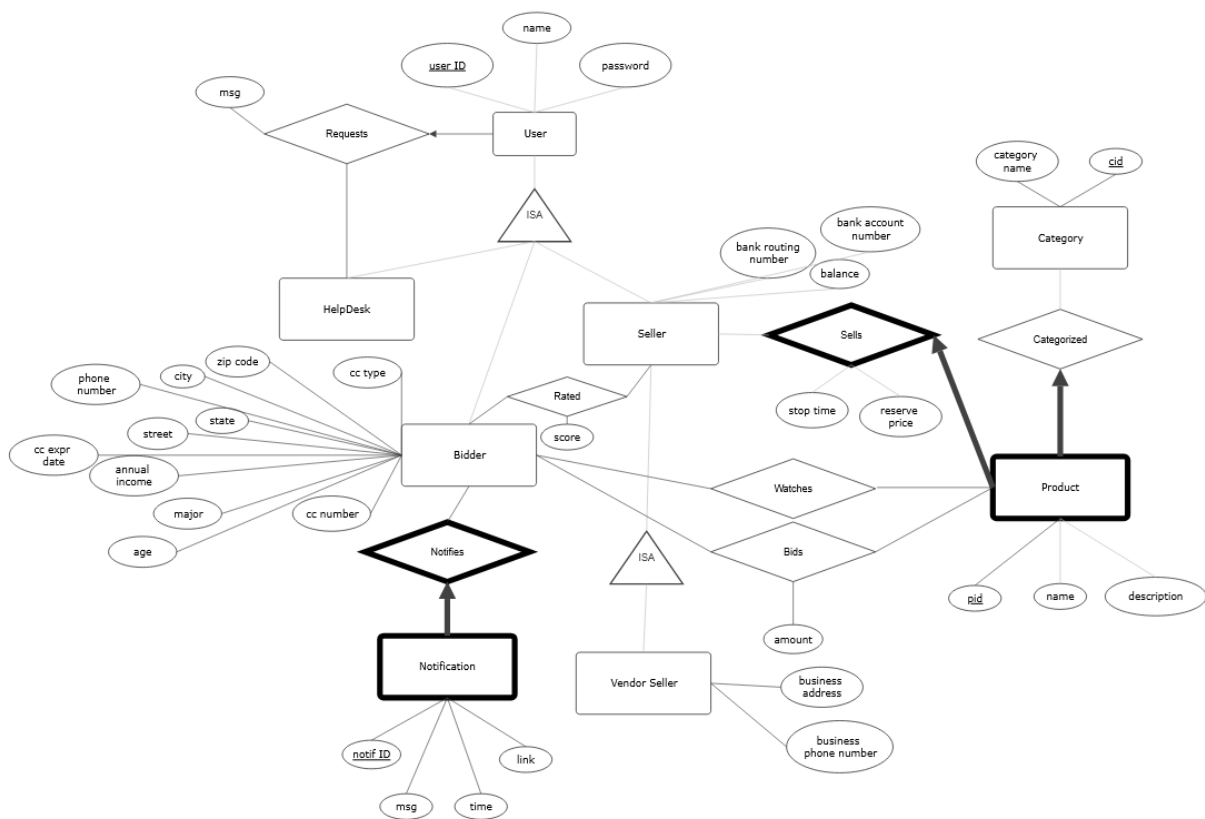


Figure 1 – A complete ER diagram

3.1 Users: Bidders, Sellers, and HelpDesk

Since all users share some common information, we can reduce redundancy and relate Bidders, Sellers, and HelpDesk to Users with an ISA relationship. Users carry the login information (with user ID as a key constraint) as well as the names. Users are related to HelpDesk by requests they make (for bidders, requests to become sellers. For sellers, new categories) with a given message (msg property). Each user can send out only one request, to prevent spam. For constraints not listed here: All Users must specialize into an inheriting entity, so there are no isolated users. Additionally, no two entities can inherit the same user (for example, an email cannot be associated with both a Bidder and a Seller).

For bidders, all associated information not found in the user entity is given directly as a property of the bidder entity: address (street, city, state, zipcode), phone number, credit card info (type, number, expiration date), major, age, and annual income. Bidders are related to Products by a Bid with a given amount (more on that in 3.2) and Watches, to their Notifications by a Notifies relationship, and to Sellers by a Rated relationship (more on that in the following paragraph).

For Sellers, extra information is similarly directly stored as a property: bank routing number, bank account number, and balance. Sellers are related to Products by a Sells relationship, with an associated stop time and reserve price. The Rated relationship determines their rating. Each Bidder can only rate a Seller once, and a Seller's average rating can be determined by a simple SQL query into the average score of all ratings including the seller's user ID.

Vendor Sellers inherit the Seller entity, with an associated business address and business phone number. All Sellers are not required to be Vendor Sellers. Additionally, although the graph implies products would be deleted when any seller is removed (and not just a vendor), we do not allow sellers to be removed. So, in practice, this constraint only applies to vendors.

HelpDesk employees also inherit users, but have no special properties or relationships of their own. This distinction is only here so that HelpDesk employees can benefit from basic user information (login details) without other types of users inheriting HelpDesk status.

3.2 Products, Bidding, Categories

Products form a weak entity with the Sells relationship, as related to Sellers. They have the key constraint pid and additional properties name and description. They are related to Bidders by Bids and Watches, and related to Categories by Categorized (wherein each Product must have exactly one Category).

Bids are represented as a relationship between Bidders and Products. They carry the amount property. We only record the latest (highest) bid between the same Product and Bidder, although any single user can place multiple bids. We can determine the current minimum acceptable next bid by finding the max amount among Bids relationships, and then adding \$1. As an additional constraint, when the current time exceeds the sale stop time, Products are automatically delisted, and new bids cannot be placed.

Categories have the key property cid, and the additional property category name. Although it cannot be illustrated, all category names MUST be unique, since duplicates do not make sense within our system. We do not allow Categories to be removed or updated once added.

3.3 Notifications and Watchlist

We include the relationship Watches, relates Bidders to Products. Whenever a new bid is placed on a Product, notifications are sent to all Bidders who are in a Watches relationship with that Product. A Watches relationship is deleted whenever the product is updated or removed, or whenever the bidder is removed. A watches relationship updates the user ID according to the Bidder's user ID.

We also include the entity Notification. A Notification contains a notif ID (key constraint), message, time, and hyperlink. A Notifies relationship is also included, as Notification is a weak entity connected to a bidder. User IDs update with the user, and we delete the notification if the Bidder is deleted.

3.4 Additional Notes

Although the given diagram is robust and includes many entities, relationships, and properties, we anticipate Our final product may differ slightly in exactly how this is implemented. In the course of coding, it becomes easier to determine the best way forward. However, we are

committed to sticking as close to this model as possible. As of the time of writing, no deficiencies have been made obvious.

4. Technology Survey

4.1 Overview of Current Web and Database Technologies

Modern web application development is dominated by a few major ecosystems; most notably are JavaScript-based stacks, like React and Node.js, and Python-Based frameworks, like Flask and Django. Recent Developer surveys continue to show Javascript and Python among the most widely used languages for back end and full-stack development. This reflects their strong community support, and rich libraries and ecosystems.

Relational Database Management Systems (RDBMSs) like PostgreSQL, MySQL, and SQLite are widely used in modern transactional applications that require structured schemas, and support for SQL. There are also popular non-relational systems such as MongoDB for certain classes of web applications, but they may not align as directly with the relational modeling and normalization principles we emphasized in earlier sections of this report.

In this section, we describe several candidate technology stacks for implementing NittanyAuction, compare them with the recommended platform (Flask, Python, PyCharm, SQLite) and justify our final selection based on technical fit, learning curve, and project constraints.

4.2 The Baseline Stack: Flask, Python, PyCharm, and SQLite

First, we will go over the stack recommended by the Course Administrators, which is again Flask, Python, PyCharm, and SQLite. This stack is representative of technologies that are widely used in industry. Each component is widely used in real applications, like Netflix, Lyft, Pinterest, and Airbnb, adding real world credibility to this stack.

Flask is a lightweight Python web framework that provides routing, request handling, and templating, but leaves most architectural decisions to the development team. This design makes flask well-suited for small and medium application sizes, where a team wants control over the application structure, and database access.

Python itself remains one of the most widely used programming languages in industry and education, particularly for back-end development and data-centric applications. PyCharm is a dedicated Python IDE that offers editor support, debugging, virtual environment management, and integration for popular Python web frameworks, which can streamline development for projects.

SQLite is a file-based relational DBMS that implements most standard SQL features, and is embedded directly into the application process. It is widely used in prototyping and embedded systems, and it provides enough functionality to support web application patterns without the overhead of managing a separate database server.

There are several advantages to this stack for NittanyAuction: Flask and SQLite make it easy to write and reason about SQL queries that reflect our ER model and normalized schema,

Flask's minimal structure allows us to implement exactly the functionality required by our Requirement Analysis, without having to adopt a large framework's conventions, and Using the recommended tools maximizes the guidance we can receive from the Teaching Staff and reduces technical risk for the project.

With advantages comes limitations, however and there are a few with this stack: Flask requires more manual wiring of components such as authentication, input validation, and admin interfaces compared to frameworks like Django, and SQLite is not optimized for larger scale production deployments, so a real world auction platform would likely require a more robust server based RDBMS such as PostgreSQL or MySQL.

4.3 Alternative Stack: Django, Python, PostgreSQL

Django is a high-level Python web framework that provides built-in support for URL routing, templating, authentication, an object-relational mapper (ORM), and an automatically generated administrative interface. Django encourages a structured, modular architecture and is widely used in industry for content-driven and transactional web applications.

PostgreSQL is a powerful open-source relational database system that supports advanced SQL features, and extensive indexing and data-type support. PostgreSQL is ranked among the most popular and fast-growing RDBMSs for web applications and Software as a Service (SaaS) platforms.

The advantages of this stack include out of the box user authentication, form handling, and administrative UI, and Django's ORM is designed to map relational schemas to Python objects, and running on PostgreSQL allows us to leverage a widely used production grade RDBMS.

The disadvantages of this stack are the higher learning curve that Django would introduce with its own project conventions, configuration patterns, and ORM abstractions, which would require additional learning time. We would also lose direct support from the Teaching Staff, so adopting Django would reduce the level of assistance we have available and increase project risk.

4.4 Alternative Stack: MongoDB, Express, React, Node.js

The MERN stack combines MongoDB, a document-oriented NoSQL database, Express.js, a minimal Node.js back-end framework, React, a component-based front-end library, and Node.js, a JavaScript runtime on the server. This all-JavaScript stack is widely used in modern web development, especially for single-page applications that emphasize client-side interactions.

React is one of the most popular front-end technologies, and Node.js has become a mainstream back-end platform, making MERN highly relevant in current industry practice. MongoDB, as a document database, allows flexible schemas and is designed to scale horizontally, which can be attractive for fast-moving teams and large systems.

Advantages of the MERN stack include simplified skill requirements, because developers can use JavaScript for both the Client and Server, simplifying skill requirements, and potentially increasing development speed. MERN is also widely covered in tutorials and bootcamps.

The most significant disadvantage to this stack is simply the lack of SQL. MongoDB does not naturally support relational concepts, like ER modeling, and normalization that are central to CMPSC431W. Additionally, adopting React, Node.js, Express, and MongoDB would significantly increase the number of technologies our team must learn, leaving less time to focus on database design and query formulation.

4.5 Alternative Stack: React/Next.js, Node.js, and PostgreSQL

Another modern pattern is to build the front-end using React and Next.js, a framework that adds server-side rendering, built-in routing, and performance optimizations to React, while Node.js is used on the server with PostgreSQL as the primary relational database. This combination is frequently used for modern SaaS and content platforms that require robust relational data storage, and modern user interfaces.

This stack has many advantages due to its strong alignment with current industry trends. Next.js with React and PostgreSQL reflects how many modern production systems are built, which would bring NittanyAuction closer to real-world architecture. Additionally PostgreSQL preserves the relational modeling and normalization concepts that are central to CMPSC 431W, while still allowing a modern front-end.

However, this stack is significantly complex, and would require mastering React, Next.js, and Node.js which may be too ambitious for a single semester project that already demands

substantial effort. Similarly with other alternative stacks, this would also lack the support of the Teaching Staff, increasing project risk.

4.6 Database Management Systems Comparison

For NittanyAuction we considered four main database options: SQLite, PostgreSQL, MySQL, and MongoDB. Recent rankings of database popularity show PostgreSQL, MySQL and SQLite all among the top relational systems in active use worldwide, while MongoDB is a leading NoSQL document store.

SQLite is lightweight, file-based, and easy to deploy, making it ideal for development, and small prototypes, but less suitable for large-scale systems. PostgreSQL offers advanced SQL support, wider extensibility, and is widely used for modern web and analytics applications. MySQL remains a popular choice for traditional web applications and hosting environments, though its feature set and licensing differ from PostgreSQL. MongoDB provides flexible document storage and can scale horizontally but does not natively support relational constraints like the RDBMSs we are used to.

Given that the NittanyAuction project is designed to reinforce RDBMS concepts, we prioritize relational DBMSs over NoSQL systems. Among the relational options, SQLite is the simplest to integrate with Flask and offers the most support.

4.7 Development Tools and IDEs

For NittanyAuctions we determined that using a professional IDE fits this project the best. For Python-based stacks PyCharm is tailored to Python development and offers strong integration with Flask and Django workflows. An alternative editor, Visual Studio Code is familiar, and widely used for web development, providing a wide range of extensions and integrated terminals for many stacks. However, given the recommendation for Python-centric tools and our decision to adopt flask and Python, PyCHarm offers a focused and supported environment for our team.

4.8 Final Recommendation

After considering several modern web and database technology combinations, we conclude that the best stack for NittanyAuction in the context of CMPSC431W is Flask with Python, and SQLite, with PyCharm as the main IDE. This selection aligns closely with the course objectives, leverages current technologies, and remains within a manageable scope for our team.

While alternative stacks such as Django with PostgreSQL, MERN, and React/Next.js with PostgreSQL are highly relevant in industry and offer attractive features, they introduce additional complexity, and reduced support that would distract from our primary focus. For these reasons, we adopt the recommended stack and use the insights from our technology survey to guide future improvements or potential migrations beyond the prototype stage.

References

Stack Overflow, “Technology – 2025 Developer Survey,” 2025. Available: <https://survey.stackoverflow.co/2025/technology>

Noble Desktop, “Best Web Development Stacks to Use in 2025,” 2025. Available: <https://www.nobledesktop.com/blog/best-web-development-stacks>

JetBrains, “Best Python Web Framework: Django, Flask, FastAPI,” PyCharm Blog, 2026. Available: <https://blog.jetbrains.com/pycharm/2025/02/django-flask-fastapi/>

DB Data, “Top 10 Most Used RDBMS Databases in 2025,” 2026. Available: <https://www.dbdata.sk/top-10-databases-in-2025/>

Threadgold Consulting, “Most Popular Database Management Software in the World (2025),” 2025. Available: <https://threadgoldconsulting.com/research/most-popular-database-management-software-2025>

SingleStore, “The Ultimate Guide to the Best SQL Database Options in 2025,” 2025. Available: <https://www.singlestore.com/blog/ultimate-guide-best-sql-database-options-2025/>

TestDriven.io, “Django vs. Flask in 2024: Which Framework to Choose,” 2019. Available: <https://testdriven.io/blog/django-vs-flask/>

Inboundsys, “Best Web Development Stack for 2025 – A Complete Guide,” 2025.
Available: <https://inboundsys.com/blog/best-web-development-stack-guide-2025>

PLANEKS, “13 Companies that Use Flask,” 2025. Available:
<https://www.planeks.net/companies-using-flask/>

Career Karma, “Top 10 Big Companies Using Flask,” 2022. Available:
<https://careerkarma.com/blog/companies-that-use-flask/>

Entri, “Discover Top Companies Using Python Flask!,” 2023. Available:
<https://entri.app/blog/top-big-companies-using-python-flask/>

QueryPlane, “SQLite in Production,” 2026. Available:
<https://queryplane.com/docs/blog/sqlite-in-production>

Nihar Daily, “SQLite in 2025: Why This ‘Simple’ DB Powers Major Apps,” 2025.
Available: <https://www.nihardaily.com/92-the-future-of-sqlite-trends-developers-must-know>

Pockit Tools (DEV Community), “The SQLite Renaissance: Why the World’s Most Deployed Database Is Taking Over Production in 2026,” 2026. Available:
https://dev.to/pockit_tools/the-sqlite-renaissance-why-the-worlds-most-deployed-database-is-taking-over-production-in-2026-3jcc

Dev Community, “Choosing Tech Stack in 2025: A Practical Guide,” 2025. Available:
<https://dev.to/dimeloper/choosing-tech-stack-in-2025-a-practical-guide-4gll>

5. Logical Database Design and Schema Normalization

5.1 Overview

In this section, the conceptual ER model of NittanyAuction is translated into a relational database schema, and then refined through normalization. The objective is to produce a schema that supports the required functionality of the platform while reducing redundancy and preserving integrity constraints. The final schema is designed to satisfy 3rd Normal Form.

The NittanyAuction platform includes three major user roles: bidders, sellers, and HelpDesk staff. Products are listed by sellers, categorized within a category hierarchy, bid on by bidders, and archived after auction completion. Additionally, the schema also supports the proposed watchlist and notification functionality described earlier.

Although the ER diagram provides the conceptual basis for the design, several changes are necessary when translating it into relations. Relationships such as Bids, Watches, Rated, Notifies, Categorized, and Requests need to be represented in the relational model. Similarly, some attributes are separated into their own relations to reduce redundancy, and to satisfy the functional dependencies given in the project description.

5.2 Mapping the ER Diagram to Relations

The ER model contains the core entity User, with subtypes Bidder, Seller, and HelpDesk through an ISA. In the relational schema, this is implemented using one relation for the supertype, and one relation for each subtype. The subtype tables use the same key as the supertype table, which also acts as the foreign key back to Users.

5.2.1 User and Role Subtypes

The entity User contains the general account information shared by all users, including: user ID, name, and password, which gives us the relation:

Users(user_id, name, password)

The ISA is then mapped onto Bidders(user_id, ...), Sellers(user_id, ...), and HelpDesk(user_id, ...). Each of these uses user_id as its primary key, which also acts as a foreign key referencing Users(user_id). This design preserves the subtype structure while avoiding duplication of shared user account information.

5.2.2 Bidder Information

The ER diagram places bidder specific information directly on the Bidder entity, including: phone number, street, city, state, zip code, credit card type, credit card number, credit card expiration date, major, age, and annual income. This leaves us with Bidders(user_id, phone_number, street, city, state, zip_code, cc_type, cc_number, cc_exp_date, major, age, annual_income). These attributes allow for bidder account management, payment processing, and market analysis as described earlier.

5.2.3 Seller and Vendor Seller Information

The Seller subtype stores financial information required to manage the transactions that happen within the system. These attributes include bank routing number, bank account number, and account balance. This information is represented by the relation:

Sellers(user_id, bank_routing_number, bank_account_number, balance).

The ER model also includes VendorSeller as a subtype of Seller. Vendor sellers require additional business information including business address and business phone number, resulting in the relation: VendorSellers(user_id, business_address, business_phone_number).

The VendorSellers relation references the Sellers relation using user_id, preserving the subtype hierarchy defined in the ER diagram.

5.2.4 HelpDesk

The ER diagram includes HelpDesk as a subtype of User without additional attributes. This becomes: HelpDesk(user_id). This relation identifies which users have administrative privileges within the NittanyAuction System.

5.2.5 Categories

The system organizes products using a category structure that allows bidders to browse listings efficiently. The ER diagram models categories with the attributes cid and category name. This produces the relation: Categories(cid, category_name). The cid attribute acts as the primary key for this relation.

5.2.6 Products and Seller Listings

The ER diagram models Product as a weak entity related to Seller through the Sells relationship. Products contain the attributes pid, name, and description. The Sells relationship includes the attributes reserve_price, and stop_time. These attributes are combined into a single relation representing product listings: Products(pid, seller_id, name, description, reserve_price, stop_time).

Here the seller_id references Sellers(user_id), preserving the relationship between sellers and the products they list. Products are also associated with categories through the Categorized relationship. This relationship is represented using: Categorized(pid, cid) which connects products to their assigned category.

5.2.7 Bids

The ER diagram models Bids as a relationship between Bidder and Product with attribute amount. This relationship is represented by the relation Bids(bidder_id, pid, amount). Here, bidder_id references Bidders(user_id), and pid references Products(pid). This relation records the current stored bid amount between a bidder and a product listing. Only the latest (highest) bid between the same bidder and product is stored.

5.2.8 Ratings

The ER diagram includes the Rated relationship between Bidder and Seller, with the attribute score. This relationship stores the score a bidder gives to a seller, and each bidder can rate a seller at most once. This relationship is represented as Rated(bidder_id, seller_id, score), where bidder_id references Bidders(user_id), and seller_id references Sellers(user_id)

5.2.9 Watchlist and Notifications

The ER diagram includes the Watches relationship between Bidder and Product, which allows bidders to follow auctions they are interested in. This relationship is represented as Watches(bidder_id, pid). The ER model also now includes the Notification entity with attributes notif_id, message, time, and link, which is represented as Notifications(notif_id, message, time, link). Notifications are associated with bidders through the Notifies(notif_id, bidder_id) relationship.

5.2.10 Requests

The ER diagram also includes a Requests relationship between Users and HelpDesk, with the attribute msg. This relationship is used to represent requests made by users to HelpDesk.

Since each user can send at most one request, this relationship is represented as

Requests(user_id, helpdesk_id, msg). Here user_id references Users(user_id), and helpdesk_id references HelpDesk(user_id). Because each user can only submit one request, user_id serves as the primary key of the Requests relation.

5.3 Initial Relational Schema Before Normalization

A direct translation of the ER diagram results in the following relations (before normalizing):

- Users(user_id, name, password)
- Bidders(user_id, phone_number, street, city, state, zip_code, cc_type, cc_number, cc_exp_date, major, age, annual_income)
- Sellers(user_id, bank_routing_number, bank_account_number, balance)
- VendorSellers(user_id, business_address, business_phone_number)
- HelpDesk(user_id)
- Categories(cid, category_name)
- Products(pid, seller_id, name, description, reserve_price, stop_time)
- Categorized(pid, cid)
- Bids(bidder_id, pid, amount)
- Rated(bidder_id, seller_id, score)
- Watches(bidder_id, pid)
- Notifications(notif_id, message, time, link)
- Notifies(notif_id, bidder_id)
- Requests(user_id, helpdesk_id, msg)

We are told in the project description that we assume two function dependencies: zip codes in address determines state and cities, and student email address determines the login password, so we will implement those next.

5.4 Functional Dependencies

We need to fulfill $\text{zip_code} \rightarrow \text{city, state}$; $\text{user_id} \rightarrow \text{password}$.

In the initial bidder relation, the attributes city and state are determined by zip_code. This creates a dependency $\text{user_id} \rightarrow \text{zip_code} \rightarrow \text{city, state}$. To remove this dependency, we need to store zip code information separately in a new relation ZipCodes(zip_code, city, state). The Bidders relation then stores only zip_code as a foreign key.

In order to satisfy $\text{user_id} \rightarrow \text{password}$, since each user has exactly one password associated with their account, the attribute password is only stored in the Users relation. This avoids repeating password data in other tables that reference users.

5.5 Normalization

First Normal Form- First Normal Form requires that every field of a relation contains only atomic values, not allowing multi-value attributes and composite attributes. The relations from the ER diagram already satisfy this because each attribute stores one value per record.

Second Normal Form- Second Normal Form requires that it is already 1NF and every non-key attribute is fully functionally dependent on the entire primary key. This requirement prevents partial dependencies where an attribute depends on only part of a composite key. For NittanyAuction, relations that use composite keys, like Watches and Bids, contain attributes that depend on the full key rather than only one component of it, so the schema satisfies the requirements of 2NF.

Third Normal Form- A relation is in Third Normal Form if it is 2NF and does not contain transitive dependencies, meaning that non key attributes should not depend on other non-key attributes. One functional dependency that we need to consider is $\text{zip_code} \rightarrow \text{city, state}$. This dependency suggests that the values of city and state are determined by the zip code rather than directly by the primary key of the bidder relation. If city and state were stored together with the bidder information, this would create a transitive dependency, so we have the scheme separate this information into its own relation: $\text{ZipCodes}(\text{zip_code}, \text{city}, \text{state})$. The Bidders relation then stores only the zip code as a foreign key. This removes the transitive dependency and ensures that the schema satisfies Third Normal Form.

5.6 Final Normalized Schema

The resulting schema has the following relations:

- Users(user_id, name, password)
- Bidders(user_id, phone_number, street, zip_code, cc_type, cc_number, cc_exp_date, major, age, annual_income)
- ZipCodes(zip_code, city, state)
- Sellers(user_id, bank_routing_number, bank_account_number, balance)
- VendorSellers(user_id, business_address, business_phone_number)
- HelpDesk(user_id)
- Categories(cid, category_name)
- Products(pid, seller_id, name, description, reserve_price, stop_time)
- Categorized(pid, cid)
- Bids(bidder_id, pid, amount)
- Rated(bidder_id, seller_id, score)
- Watches(bidder_id, pid)
- Notifications(notif_id, message, time, link)
- Notifies(notif_id, bidder_id)
- Requests(user_id, helpdesk_id, msg)

This schema preserves the ER model's design while reducing redundancy and satisfying the required functional dependencies.

5.7 SQL Schema

```
CREATE TABLE Users (  
    user_id CHAR(50) PRIMARY KEY,  
    name CHAR(50) NOT NULL,  
    password CHAR(50) NOT NULL  
);
```

```
CREATE TABLE ZipCodes (  
    zip_code CHAR(10) PRIMARY KEY,  
    city CHAR(30) NOT NULL,  
    state CHAR(20) NOT NULL  
);
```

```
CREATE TABLE Bidders (  
    user_id CHAR(50) PRIMARY KEY,  
    phone_number CHAR(15),  
    street CHAR(100),  
    zip_code CHAR(10) NOT NULL,  
    cc_type CHAR(20),  
    cc_number CHAR(20),  
    cc_exp_date DATE,  
    major CHAR(50),  
    age INTEGER,  
    annual_income REAL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    FOREIGN KEY (zip_code) REFERENCES ZipCodes(zip_code)  
        ON UPDATE CASCADE  
);
```

```
CREATE TABLE Sellers (  
    user_id CHAR(50) PRIMARY KEY,  
    bank_routing_number CHAR(9) NOT NULL,  
    bank_account_number CHAR(20) NOT NULL,  
    balance REAL NOT NULL DEFAULT 0,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE  
);
```

```

CREATE TABLE VendorSellers (
    user_id CHAR(50) PRIMARY KEY,
    business_address CHAR(100),
    business_phone_number CHAR(15),
    FOREIGN KEY (user_id) REFERENCES Sellers(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

```

```

CREATE TABLE HelpDesk (
    user_id CHAR(50) PRIMARY KEY,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE
);

```

```

CREATE TABLE Categories (
    cid INTEGER PRIMARY KEY,
    category_name CHAR(40) NOT NULL UNIQUE
);

```

```

CREATE TABLE Products (
    pid INTEGER PRIMARY KEY,
    seller_id CHAR(50) NOT NULL,
    name CHAR(60) NOT NULL,
    description CHAR(255),
    reserve_price REAL NOT NULL,
    stop_time DATE NOT NULL,
    FOREIGN KEY (seller_id) REFERENCES Sellers(user_id)
        ON UPDATE CASCADE
        ON DELETE RESTRICT
);

```

```

CREATE TABLE Categorized (
    pid INTEGER PRIMARY KEY,
    cid INTEGER NOT NULL,
    FOREIGN KEY (pid) REFERENCES Products(pid)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (cid) REFERENCES Categories(cid)
        ON UPDATE CASCADE
);

```

```

        ON DELETE RESTRICT
    );

CREATE TABLE Bids (
    bidder_id CHAR(50) NOT NULL,
    pid INTEGER NOT NULL,
    amount REAL NOT NULL,
    PRIMARY KEY (bidder_id, pid),
    FOREIGN KEY (bidder_id) REFERENCES Bidders(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (pid) REFERENCES Products(pid)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CHECK (amount > 0)
);

CREATE TABLE Rated (
    bidder_id CHAR(50) NOT NULL,
    seller_id CHAR(50) NOT NULL,
    score INTEGER NOT NULL,
    PRIMARY KEY (bidder_id, seller_id),
    FOREIGN KEY (bidder_id) REFERENCES Bidders(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (seller_id) REFERENCES Sellers(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CHECK (score BETWEEN 1 AND 5)
);

CREATE TABLE Watches (
    bidder_id CHAR(50) NOT NULL,
    pid INTEGER NOT NULL,
    PRIMARY KEY (bidder_id, pid),
    FOREIGN KEY (bidder_id) REFERENCES Bidders(user_id)
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    FOREIGN KEY (pid) REFERENCES Products(pid)
        ON UPDATE CASCADE
        ON DELETE CASCADE

```

);

```
CREATE TABLE Notifications (  
    notif_id INTEGER PRIMARY KEY,  
    message CHAR(255) NOT NULL,  
    time DATE NOT NULL,  
    link CHAR(255)
```

);

```
CREATE TABLE Notifies (  
    notif_id INTEGER PRIMARY KEY,  
    bidder_id CHAR(50) NOT NULL,  
    FOREIGN KEY (notif_id) REFERENCES Notifications(notif_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    FOREIGN KEY (bidder_id) REFERENCES Bidders(user_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE
```

);

```
CREATE TABLE Requests (  
    user_id CHAR(50) PRIMARY KEY,  
    helpdesk_id CHAR(50) NOT NULL,  
    msg CHAR(255) NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES Users(user_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE,  
    FOREIGN KEY (helpdesk_id) REFERENCES HelpDesk(user_id)  
        ON UPDATE CASCADE  
        ON DELETE CASCADE
```

);

5.8 Summary

The logical database design for NittanyAuction was obtained by translating the conceptual ER diagram into relational tables and applying normalization techniques. The resulting schema preserves the ER model's design while satisfying functional dependencies, and it satisfies 3rd Normal Form, providing a good foundation for the next phase of the project.